Seminar 7 PC

A. Alocarea dinamica a memoriei in C

Clase de memorare in C

Clasa de memorare arata cand, cum si unde se aloca memorie pentru o variabila (vector). Orice variabila C are o clasa de memorare care rezulta fie dintr-o declaratie explicita, fie implicit din locul unde este definita variabila.

Exista trei moduri de alocare a memoriei, dar numai doua corepund unor clase de memorare:

- **Static**: memoria este alocata la compilare in segmentul de date din cadrul programului si nu se mai poate modifica in cursul executiei. Variabilele externe, definite in afara functiilor, sunt implicit statice, dar pot fi declarate “static” si variabile locale, definite in cadrul functiilor.

- **Automat**: memoria este alocata automat, la activarea unei functii, in zona stiva alocata unui program si este eliberata automat la terminarea functiei. Variabilele locale unui bloc (unei functii) si argumentele formale sunt implicit din clasa “auto”.

- **Dinamic**: memoria se aloca la executie in zona “heap” alocata programului, dar numai la cererea explicita a programatorului, prin apelarea unor functii de biblioteca (malloc, calloc, realloc). Memoria este eliberata numai la cerere, prin apelarea functiei “free”. Variabilele dinamice nu au nume si deci nu se pune problema clasei de memorare (atribut al variabilelor cu nume).

Variabilele statice pot fi initializate numai cu valori constante (pentru ca se face la compilare), dar variabilele “auto” pot fi initializate cu rezultatul unor expresii (pentru ca se face la executie). Toate variabilele externe (si statice) sunt automat initializate cu valori zero (inclusiv vectorii).

Cantitatea de memorie alocata pentru variabilele cu nume rezulta din tipul variabilei si din dimensiunea declarata pentru vectori. Memoria alocata dinamic este specificata explicit ca parametru al functiilor de alocare.

O a treia clasa de memorare este clasa “register” pentru variabile carora li se aloca registre ale procesorului si nu locatii de memorie, pentru un timp de acces mai bun.

Memoria neocupata de datele statice si de instructiunile unui program este impartita intre stiva si “heap”. Consumul de memorie “stack” (stiva) este mai mare in programele cu functii recursive si numar mare de apeluri recursive, iar consumul de memorie “heap” este mare in programele cu vectori si matrice alocate (si realocate) dinamic.

Functii de alocare si eliberare a memoriei

Aceste functii standard sunt declarate in fisierele <alloc.h> si <stdlib.h>. Cele trei functii de alocare au ca **rezultat adresa zonei de memorie alocate (de tip void *)** si ca argument comun **dimensiunea zonei de memorie alocate (de tip size_t).** Daca cererea de alocare nu poate fi satisfacuta pentru ca nu mai exista un bloc continuu de dimensiunea solicitata, atunci functiile de alocare au rezultat NULL.

La apelarea functiilor de alocare se folosesc:

- **operatorul "sizeof"** pentru a determina numarul de octeti necesar unui tip de date (variabile);

- **operatorul de conversie "cast"** pentru adaptarea adresei primite la tipul datelor memorate la adresa respectiva (conversie necesara atribuirii intre pointeri de tipuri diferite).

Exemple:

```
char *str = (char*) malloc(30);          // aloca memorie pentru 30 de caractere

int *a =  (int *) malloc ( n * sizeof(int));  // aloca memorie pentru n numere intregi
```

Alocarea de memorie pentru un vector si initializarea zonei alocate cu zerouri se poate face cu **functia "calloc".**

Exemplu:

```
int *a= (int*) calloc (n, sizeof(int) );    // aloca memorie pentru n numere intregi
```

Realocarea unui vector care creste (sau scade) fata de dimensiunea estimata anterior se poate face cu **functia "realloc",** care primeste adresa veche si noua dimensiune si intoarce noua adresa:

```
a = (int *) realloc (a, 2*n* sizeof(int));          // dublare dimensiune anterioara (n)
```

In exemplul anterior noua adresa este memorata tot in variabila pointer "a", inlocuind vechea adresa (care nu mai este necesara si nici nu mai trebuie folosita).

Functia "realloc" realizeaza urmatoarele operatii:

- aloca o zona de dimensiunea specificata ca al doilea argument

- copiaza la noua adresa datele de la adresa veche (primul argument al functiei)

- elibereaza memoria de la adresa veche.

Functia "free" are ca argument o adresa (un pointer) si elibereaza zona de la adresa respectiva (alocata prin apelul unei functii "...alloc"). Dimensiunea zonei nu mai trebuie specificata deoarece este memorata la inceputul zonei alocate (de catre functia de alocare).

Vectori alocati dinamic

Structura de vector are avantajul simplitatii si economiei de memorie fata de alte structuri de date folosite pentru memorarea unei colectii de date intre care exista anumite relatii.

Intre cerinta de dimensionare constanta a unui vector si generalitatea programelor care folosesc astfel de vectori exista o contradictie. De cele mai multe ori programele pot afla (din datele citite) dimensiunile vectorilor cu care lucreaza si deci pot face o alocare dinamica a memoriei pentru acesti vectori. Aceasta este o solutie mai flexibila, care foloseste mai bine memoria disponibila si nu impune limitari arbitrare asupra utilizarii unor programe. In limbajul C nu exista practic nici o diferenta intre utilizarea unui vector cu dimensiune fixa si utilizarea unui vector alocat dinamic, ceea ce incurajeaza si mai mult utilizarea unor vectori cu dimensiune variabila.

De observat ca **nu orice vector cu dimensiune constanta este un vector static; un vector definit intr-o functie (alta decat "main") nu este static deoarece nu ocupa memorie pe toata durata de executie a programului, desi dimensiunea sa este stabilita la scrierea programului**.

Exemplul urmator arata cum se poate defini si utiliza un vector alocat dinamic:

```
void main() {
        int n,i;   int * a;                         // adresa vector alocat dinamic
        printf ("n="); scanf ("%d", &n);            // dimensiune vector
        a=(int *) calloc (n,sizeof(int));           // a=(int*) malloc (n*sizeof(int));
        printf ("componente vector: \n");
        for (i=0;i<n;i++)
          scanf ("%d", &a[i]);              // sau   scanf ("%d", a+i);
        for (i=0;i<n;i++)                           // afisare vector
          printf ("%d ",a[i]);
}
```

Exista si cazuri in care datele memorate intr-un vector rezulta din anumite prelucrari, iar numarul lor nu poate fi cunoscut de la inceputul executiei. Un exemplu poate fi un vector cu toate numerele prime mai mici ca o valoare data. In acest caz se poate recurge la o **realocare dinamica a memoriei**.

In *exemplul* urmator se citeste un numar necunoscut de valori intregi intr-un vector extensibil:

```
#include <stdio.h>
#include <stdlib.h>

#define INCR 5    // cu cat creste vectorul la fiecare realocare
void main() {
 int n,i,m ;
 float x, * v;                    // v = adresa vector
 n=INCR; i=-1;
 v = (float *) malloc (n*sizeof(float));       // dimensiune initiala vector

 while (scanf("%f", &x) != EOF) {
   if (++i == n) {                             // daca este necesar
     n = n + INCR;                    // creste dimensiune vector
     v = (float *) realloc (v, n*sizeof(float));
   }
   v[i] = x;            // memorare in vector numar citit
 }

 for (i=0; i<n; i++) // afisare vector
   printf ("%.2f ", v[i]);
}
```

Din exemplele anterioare lipseste eliberarea memoriei alocate pentru vectori, dar fiind vorba de un singur vector alocat in functia "main" si necesar pe toata durata de executie, o eliberare finala este inutila. Inainte de executia unui nou program memoria "heap" este reinitializata (se anuleaza automat toate alocarile facute de programul anterior). Eliberarea explicita poate fi necesara pentru vectori de lucru, folositi numai in anumite secvente de program (sau functii).

Realocarea repetata de memorie poate conduce la fragmentarea memoriei "heap", adica la crearea unor blocuri de memorie libere dar neadiacente si prea mici pentru a mai fi reutilizate ulterior. De aceea, politica de realocare pentru un vector este uneori dublarea capacitatiii sale anterioare.

Vectori de pointeri la date alocate dinamic

Pentru memorarea mai multor siruri de caractere, de lungimi foarte diferite, este eficient sa alocam dinamic memoria pentru fiecare sir, in functie de lungimea sa. Adresele acestor siruri sunt in general dispersate in memorie. Mai corect, programatorul nu poate controla modul de alocare si nici adresele furnizate prin apeluri succesive ale functiei “malloc” (sau “calloc”).

Reunirea adreselor sirurilor alocate dinamic se poate face simplu intr-un vector de pointeri catre aceste siruri.

Exemplu:

```
 // citire lista siruri, cu vector de pointeri
int readstr ( char * vp[] ) {
  int n = 0;   char * p, sir[80];
  while ( scanf ("%s", sir) > 0) {              // citire la adresa “sir”
    p= (char*) malloc (strlen(sir)+1);          // aloca memorie in heap pentru sir
    strcpy( p,sir);                             // copiaza sir citit in heap
    vp[n] = p;  n++;                            // memoreaza adresa sir in vector
  }
  return n;                                 // cate siruri s-au citit
}
  // afisare siruri reunite intr-un vector de pointeri
void printstr ( char * vp[], int n) {
 int i;
 for(i=0;i<n;i++)
    printf ("%s\n",vp[i]);
}
  // utilizare functii - vector de pointeri cu dimensiune fixa
void main ( ) {
  char * list[100];  int n;
```

```
   n = readstr(list);
   printstr (list, n);
}
```

Un vector de pointeri alocat dinamic (pentru n siruri) se declara si se foloseste astfel:

```
void main () {
  int n;  char ** list;                    // adresa vector de pointeri
  printf ("n="); scanf ("%d",&n);          // dimensiune vector de pointeri
  list = (char**) calloc ( n, sizeof (char*));
  readstr (list,n);  printstr (list,n);
}
```

Argumente in linia de comanda

Functia "main" poate avea doua argumente, prin care primeste date transmise prin linia de comanda ce lanseaza programul in executie. Sistemul de operare analizeaza linia de comanda, extrage cuvintele din linie (siruri separate prin spatii albe), aloca memorie pentru aceste cuvinte si introduce adresele lor intr-un vector de pointeri (alocat dinamic).

Primul argument al functiei "main" este dimensiunea vectorului de pointeri (de tip "int"), iar la doilea argument este adresa vectorului de pointeri (deci un pointer). Exemplu:

```
void main ( int argc, char * argv[]) {
  int i;
  for (i=1;i<n;i++)                  // nu se afiseaza si argv[0]
    printf ("%s ", argv[i]);
}
```

Primul cuvant, cu adresa in argv[0], este chiar numele programului executat (numele fisierului ce contine programul executabil), iar celelalte cuvinte din linie sunt date initiale pentru program (nume de fisiere folosite de program, optiuni de lucru diverse).

Prelucrarea argumentelor din linia de comanda este de obicei secventiala si deci putem renunta la indici in referire la componentele vectorului de pointeri. Exemplu:

```
void main ( int argc, char ** argv) {
  while (argc --)
    printf ("%s ", *argv++);
}
```

Matrice alocate dinamic

Alocarea dinamica pentru o matrice este importanta deoarece:

- Foloseste economic memoria si evita alocari acoperitoare, estimative.
- Permite matrice cu linii de lungimi diferite.
- Reprezinta o solutie buna la problema argumentelor de functii de tip matrice.

Daca programul poate afla numarul efectiv de linii si de coloane al unei matrice (cu dimensiuni diferite de la o executie la alta), atunci se va aloca memorie pentru un vector de pointeri (functie de numarul liniilor) si apoi se va aloca memorie pentru fiecare linie (functie de numarul coloanelor) cu memorarea adreselor liniilor in vectorul de pointeri. O astfel de matrice se poate folosi la fel ca o matrice declarata cu dimensiuni constante.

Exemplu:

```
void main () {
  int ** a; int i,j,nl,nc;
  printf ("nr. linii="); scanf ("%d",&nl);
  printf ("nr. col. ="); scanf ("%d",&nc);
```

```
  a = (int**) malloc (nl*sizeof(int*));          // alocare pentru vector de pointeri
  for (i=0; i<n;i++)
    a[i] = (int*) calloc (nc, sizeof(int));      // alocare pentru o linie si initializare
    // completare diagonala matrice unitate
  for (i=0;i<nl;i++)
    a[i][i]=1;                                   // a[i][j]=0 pentru i != j
   // afisare matrice
  printmat (a,nl,nc);
}
```

Functia de afisare a matricei se poate defini astfel :

```
void printmat (int ** a, int nl,int nc) {
  for (i=0;i<nl;i++) {
    for (j=0;j<nc;j++)
      printf ("%2d", a[i][j]);
    printf("\n");
  }
}
```

Notatia a[i][j] este interpretata astfel pentru o matrice alocata dinamic:

a[i] contine un pointer (o adresa b)

b[j] sau b+j contine intregul din pozitia "j" a vectorului cu adresa "b".

Functia "printmat" data anterior nu poate fi apelata dintr-un program care declara argumentul efectiv ca o matrice cu dimensiuni constante. *Exemplul* urmator este corect sintactic dar **nu se executa corect**:

```
void main () {
```

```
    int x [2][2]={{1,2},{3,4}};       // o matrice patratica cu 2 linii si 2 coloane
     printmat ( (int**)x, 2, 2);
  }
```

Explicatia este interpretarea diferita a continutului zonei de la adresa aflata in primul argument: functia “printmat” considera ca este adresa unui vector de pointeri (int * a[]). iar programul principal considera ca este adresa unui vector de vectori (.int x[][2]).

Se poate defini si o functie pentru alocarea de memorie la executie pentru o matrice. Exemplu:

```
int * * newmat ( int nl, int nc) {                    // rezultat adresa matrice
  int i;
  int ** p=(int **) malloc (nl*sizeof (int*));
  for (i=0;i<n;i++)
    p[i] =(int*) calloc (nc,sizeof (int));
  return p;
}
```

B. FUNCȚII CU NUMĂR VARIABIL DE ARGUMENTE. PROGRAMARE GENERICĂ

1. Scrieți o **funcție cu număr variabil de parametri** care să concateneze mai multe șiruri de caractere.

Rezolvare:

Prima variantă de rezolvare constă în implementarea unei funcții cu număr variabil de parametri având ca parametru fix numărul șirurilor de caractere care vor fi concatenate:

char* concat1(**int** n,...)

Un exemplu de apel corect al acestei funcții este s = concat1(3, "Test", " ", "apel"), în urmă căruia se va obține șirul s = "Test apel". Atenție, înainte de terminarea programului, zona de memorie alocată pentru șirul de caractere **s** trebuie eliberată!

Pentru a putea să alocăm dinamic un șir **aux** care va fi obținut prin concatenarea șirurilor date, este necesar să cunoaștem lungimea sa (notată cu **lt**). Astfel, vom parcurge o dată lista parametrilor variabili **siruri_1** pentru a determina suma lungimilor tuturor șirurilor respective, adică valoarea **lt**. Deoarece o astfel de listă poate fi parcursă o singură dată, iar noi va trebui să o parcurgem încă o dată pentru a concatena șirurile respective, vom salva o copie a sa în lista **siruri_2**.

```
char* concat1(int n, ...)
{
    char *aux, *sir;
    int i, lt;
    va_list siruri_1, siruri_2;
    va_start(siruri_1, n);
    va_copy(siruri_2, siruri_1);
    lt = 0;
    for(i = 0; i < n; i++)
    {
        sir = va_arg(siruri_1, char*);
        lt = lt + strlen(sir);
    }
    va_end(siruri_1);
    aux = (char *)malloc(lt + 1);
    strcpy(aux, "");
    for(i = 0; i < n; i++)
    {
        sir = va_arg(siruri_2, char*);
```

```
        strcat(aux, sir);
    }
    va_end(siruri_2);
    return aux;
}
```

A doua variantă de rezolvare constă în implementarea unei funcții care nu mai are ca parametru numărul șirurilor care vor fi concatenate, ci sfârșitul listei parametrilor variabili va fi marcat prin pointerul NULL:

char* concat2(char *s,...)

În acest caz, parametrul fix **s** reprezintă chiar primul șir dintre cele pe care trebuie să le concatenăm. Un exemplu de apel corect al acestei funcții este s = concat2("Alt", " ", "apel", NULL), în urmă căruia se va obține șirul s = "Alt apel".

În plus, *pentru a elimina dubla parcurgere a listei parametrilor variabili, vom realoca dinamic* șirul **aux** în care construim șirul rezultat prin concatenarea șirurilor date.

```
char* concat2(char *s, ...)
{
    char *aux, *sir;
    va_list siruri;
    if(s == NULL)
    {
        aux = (char *)malloc(1);
        strcpy(aux, "");
        return aux;
    }
    va_start(siruri, s);
    aux = (char *)malloc(strlen(s) + 1);
    strcpy(aux, s);
    while((sir = va_arg(siruri, char*)) != NULL)
    {
        aux = realloc(aux, strlen(aux) + strlen(sir) + 1);
        strcat(aux, sir);
    }
    va_end(siruri);
    return aux;
}
```

2. Scrieți o **funcție cu număr variabil de parametri** care să caute un cuvânt dat în mai multe fișiere text. Funcția va scrie într-un fișier text, al cărui nume este dat ca parametru, numerele de ordine ale liniilor pe care apare cuvântul respectiv în fiecare fișier text dat.

Rezolvare:

Un posibil antet al funcției cerute este următorul:

void cautare_cuvant(**char** *cuv, **char** *nfout,...)

Parametrul fix **cuv** reprezintă cuvântul pe care îl vom căuta, parametrul fix **nfout** reprezintă numele fișierului text în care se vor scrie rezultatele căutării, iar lista parametrilor variabili va conține numele fișierelor text în care va fi căutat cuvântul respectiv (lista se va termina cu pointerul NULL). De exemplu, prin apelul cautare_cuvant("lac","rez.txt", "eminescu.txt","bacovia.txt",NULL) se va căuta cuvântul **lac** în fișierele text *eminescu.txt* și *bacovia.txt*, iar rezultatul căutării va fi scris în fișierul text *rez.txt*.

În cadrul funcției, preluăm, pe rând, numele fiecărui fișier text din lista parametrilor variabili în șirul de caractere nfin și efectuăm următoarele operații:

- deschidem fișierul curent folosind variabila fin;
- parcurgem fișierul linie cu linie și căutăm cuvântul dat pe linia curentă (șirul de caractere linie);
- dacă linia curentă conține cuvântul căutat cuv, atunci scriem numărul de ordine lcrt al liniei curente în fișierul de ieșire fout, marcăm faptul că am găsit cuvântul în fișierul current (folosind variabila g) și întrerupem căutarea pe linia curentă (de ce?);
- după ce am terminat de parcurs fișierul curent, verificăm dacă nu am găsit cuvântul căutat în el (adică testăm dacă g este 0 sau nu) și, în caz afirmativ, scriem un mesaj corespunzător;
- închidem fișierul curent.

```
void cautare_cuvant(char *cuv, char *nfout, ...)
{
   FILE *fin, *fout;
   char *nfin, *p;
   int lcrt, g;
   char linie[1001];
   va_list fisiere;
   va_start(fisiere, nfout);
   fout = fopen(nfout, "w");
   while((nfin = va_arg(fisiere, char*)) != NULL)
   {
      fin = fopen(nfin, "r");
      fprintf(fout, "Fisierul %s\n", nfin);
      fprintf(fout, "Liniile: ");
```

```
        lcrt = 1;
        g = 0;
        while(fgets(linie, 1001, fin) != NULL)
        {
          p = strtok(linie, " :;,.?!\n");
          while(p != NULL)
          {
            if(strcmp(p, cuv) == 0)
            {
            fprintf(fout, "%d ", lcrt);
            g = 1;
            break;
          }
      p = strtok(NULL, " :;,.?!\n");
      }
      lcrt++;
    }
    if(g == 0)
      fprintf(fout, "-");
    fprintf(fout, "\n\n");
    fclose(fin);
    }
    va_end(fisiere);
    fclose(fout);
}
```

Pentru a nu complica prea mult codul, nu am analizat posibilele erori care pot să apară:
-lista parametrilor variabili este vidă (deci nu se indică nici un fișier text în care să se efectueze căutarea),
-un anumit fișier text în care trebuie efectuată căutarea nu există,
-fișierul de ieșire nu poate fi creat etc.
Modificați codul sursă astfel încât erorile indicate mai sus să fie tratate!

3. Scrieți o **funcție cu număr variabil de parametri** care să sorteze mai multe mai multe tablouri unidimensionale formate din numere întregi pe baza unui criteriu comun.

Rezolvare:

Vom sorta tablourile date utilizând funcția qsort din biblioteca stdlib.h, deci criteriul comun de sortare trebuie implementat printr-o funcție comparator de tipul **int (*cmp)(const void*,const void*).**

Pentru a simplifica antetul funcției cerute, putem defini mai întâi un tip de date Comparator prin **typedef int (*Comparator)(const void*,const void*)**. Astfel, un posibil antet al funcției este următorul:

```
void sortare(Comparator cmp,...)
```

Lista parametrilor variabili va conține numele tablourilor pe care dorim să le sortăm, împreună cu numărul lor de elemente, iar sfârșitul listei va fi marcat prin valoarea 0. De exemplu, pentru a sorta crescător tablourile **a** și **b**, având **na** și **nb** elemente, vom proceda astfel:

- definim o funcție comparator cmpCrescator:

```
int cmpCrescator(const void* a, const void* b)
{
   int va = *(int *)a;
   int vb = *(int *)b;
   if (va < vb) return -1;
   if (va > vb) return +1;
   return 0;
}
```

- apelăm funcția prin sortare(cmpCrescator, na, a, nb, b, 0):

```
void sortare(Comparator cmp, ...)
{
   int i, n, *v;
   va_list vectori;
   va_start(vectori, cmp);
   do
   {
     n = va_arg(vectori, int);
     if(n != 0)
     {
       v = va_arg(vectori, int*);
       qsort(v, n, sizeof(int), cmp);
     }
   } while(n != 0);
   va_end(vectori);
}
```

4. Scrieți **o funcție cu număr variabil de parametri** care să returneze maximul dintre mai multe numere întregi. Folosind apeluri utile ale funcției definite anterior, scrieți o funcție care să afișeze mesajul DA în cazul în care 4 numere întregi a, b, c și d îndeplinesc condiția $a \leq b \leq c \leq d$ sau mesajul NU în caz contrar.

Rezolvare:

Considerând faptul că parametrul fix n reprezintă numărul parametrilor variabili ai funcției, vom extrage pe rând câte un număr întreg din lista parametrilor variabili și vom calcula maximul cerut:

```
int maxim(int n,...)
{
   int max, val, i;
   va_list valori;
   va_start(valori, n);
   max = INT_MIN;
   for(i = 0; i < n; i++)
   {
     val = va_arg(valori, int);
     if(val > max)
       max = val;
   }
   va_end(valori);
   return max;
}
```

Pentru a verifica faptul că 4 numere întregi îndeplinesc condiția cerută, trebuie să verificăm dacă sunt îndeplinite simultan următoarele condiții:

- maximul dintre toate cele patru numere este al patrulea număr;
- maximul dintre primele trei numere este al treilea număr;
- maximul dintre primele două numere este al doilea număr.

```
void testare(int a, int b, int c, int d)
{
if(maxim(4, a, b, c, d) == d && maxim(3, a, b, c) == c && maxim(2, a, b) == b)
   printf("DA");
else
   printf("NU");
}
```

5. Scrieți **o funcție generică** care să calculeze câte elemente dintr-un tablou unidimensional sunt egale cu o valoare dată.

Rezolvare:

Algoritmul de rezolvare a problemei este unul simplu, respectiv se parcurge tabloul element cu element și se incrementează cu 1 valoarea unei variabile de tip contor în momentul în care elementul curent este egal cu valoarea dată. Din punct de vedere al implementării, problema este mai dificilă, deoarece trebuie comparate două valori ale căror tipuri de date nu sunt cunoscute, ele fiind accesate prin intermediul unor **pointeri generici (de tip void*).** Din acest motiv, pointerii respectivi nu pot fi dereferențiați și, în consecință, nu poate fi utilizat direct operatorul de testare a egalității (==). Totuși, în cazul funcțiilor generice care manipulează tablouri, trebuie să cunoaștem dimensiunea în octeți a unui element pentru a putea simula aritmetica pointerilor (pointerii generici nu au aritmetică!). Astfel, putem folosi funcția **memcmp** pentru a compara la nivel de octet conținutul unor zone de memorie ale căror adrese și dimensiuni în octeți sunt cunoscute, în ipoteza în care ele au același tip de date:

```
int num1(const void *x, const void *tablou, int nr_elem, int dim_elem)
{
   int i, nrx;
   char *p;
   nrx = 0;
   for(i = 0; i < nr_elem; i++)
   {
     p = (char *)tablou + i*dim_elem;
     if(memcmp(p, x, dim_elem) == 0)
       nrx++;
   }
   return nrx;
}
```

Considerând **v** un tablou format din **n** numere întregi și **x** o variabilă de tip întreg, putem apela funcția **num1** de mai sus astfel: nrap = num1(&x, v, n, **sizeof**(**int**)).

Totuși, funcția num1 de mai sus nu va furniza rezultatul corect în cazul în care elementele tabloului sunt pointeri (de exemplu, în cazul unor tablouri de șiruri de caractere), deoarece se vor compara adresele între ele, ci nu valorile memorate la adresele respective! Din acest motiv, trebuie să utilizăm pentru compararea valorilor o funcție comparator, pe care o vom transmite funcției generice utilizând un pointer către o funcție (așa cum am procedat, de exemplu, în cazul funcției de bibliotecă qsort):

```
int num2(const void *x, const void *tablou, int nr_elem, int dim_elem,
int (*cmp)(const void*, const void*))
{
   int i, nrx;
   char *p;
   nrx = 0;
   for(i = 0; i < nr_elem; i++)
   {
     p = (char *)tablou + i*dim_elem;
     if(cmp(p, x) == 0)
       nrx++;
   }
   return nrx;
}
```

Considerând **v** un tablou format din **n** numere întregi și **x** o variabilă de tip întreg, putem apela funcția **num2** prin nrap = num2(&x, v, n, **sizeof(int)**, cmpIntregi), unde funcția comparator **cmpIntregi** poate fi definită astfel:

```
int cmpIntregi(const void *a , const void *b)
{
   int va = *(int *)a;
   int vb = *(int *)b;
   if(va < vb) return -1;
   if(va > vb) return 1;
   return 0;
}
```

Dacă **v** ar fi un tablou format din **n** șiruri de caractere și **s** un șir de caractere, putem apela funcția **num2** prin nrap = num2(s, v, n, **sizeof(char*)**, cmpSiruri), unde funcția comparator **cmpSiruri** poate fi definită astfel:

```
int cmpSiruri(const void *s , const void *t)
{
   return strcmp(*(char **)s , (char *)t);
}
```

Observați cu atenție modurile diferite în care au fost dereferențiați cei 2 pointeri către șiruri de caractere! De ce a fost necesar acest lucru?

6. Scrieți **o funcție generică** care să furnizeze adresa unui element cu valoare maximă dintr-un tablou unidimensional. Criteriul de comparație dintre elementele tabloului va fi implementat printr-o funcție comparator de tipul celei utilizate în funcția qsort din stdlib.h (de exemplu, funcțiile cmpIntregi și cmpSiruri de mai sus).

Rezolvare:

În cadrul funcției vom implementa algoritmul standard pentru determinarea valorii maxime dintr-un tablou unidimensional, însă nu vom lucra cu valorile elementelor din tablou, ci cu adresele lor. Astfel, variabila de tip pointer **pcrt** va conține adresa elementului curent din tablou, iar variabila **pmax** va conține adresa elementului cu valoare maximă găsit până la momentul respectiv. Pentru compararea valorilor de la adresele **pcrt** și **pmax** vom folosi funcția comparator, transmisă funcției generice printr-un pointer către o funcție.

```
void* maxim(const void *tablou, int nr_elem, int dim_elem,
int (*cmp)(const void*, const void *))
{
   int i;
   char *pcrt, *pmax;
   if(nr_elem == 0)
     return NULL;
   pmax = (char *)tablou;
   for(i = 1; i < nr_elem; i++)
   {
     pcrt = (char *)tablou + i*dim_elem;
     if(cmp(pcrt, pmax) > 0)
       pmax = pcrt;
   }
   return pmax;
}
```

Considerând un tablou unidimensional **a** format din **n** numere întregi și **pmax** o variabilă de tip pointer la int, putem afișa toate pozițiile pe care apare valoarea maximă **vmax** în tabloul **a** astfel:

```
pmax = maxim(a, n, sizeof(int), cmpIntregi);
vmax = *pmax;
while(pmax != NULL && *pmax == vmax)
{
   printf("Valoarea maxima %d apare pe pozitia %d!\n", vmax, pmax - a);
   pmax = maxim(pmax+1, n-(pmax-a)-1, sizeof(int), cmpIntregi);
}
```

7. Scrieți o **funcție generică** care să implementeze *algoritmul de căutare binară* pentru un tablou unidimensional sortat crescător în raport cu o anumită relație de ordine. Funcția va returna adresa unui element din tablou care este egal cu valoarea căutată sau pointerul NULL dacă valoarea căutată nu se găsește în tablou.

Rezolvare:

În cadrul funcției vom implementa algoritmul de căutare binară, utilizând direct adresele elementelor din tablou. Pentru compararea valorii căutate (transmisă funcției generice tot printr-o adresă) și valoarea elementului curent din tablou vom folosi funcția comparator **cmp**, în care trebuie să ținem cont de aceeași relație de ordine în raport cu care sunt sortate crescător elementele tabloului!

```
void* cbin(const void *x, const void *tablou, int nr_elem, int dim_elem,
int (*cmp)(const void* , const void *))
{
   char *pcrt;
   int st, dr, mij;
   st = 0;
   dr = nr_elem - 1;
   while(st <= dr)
   {
      mij = st + (dr-st)/2;
      pcrt = (char *)tablou + mij*dim_elem;
      if(cmp(x, pcrt) == 0)
        return pcrt;
      if(cmp(x, pcrt) < 0)
        dr = mij - 1;
      else
        st = mij + 1;
   }
   return NULL;
}
```

Modificați funcția de mai sus astfel încât să se verifice faptul că elementele tabloului sunt sortate crescător în raport cu relația de ordine implementată în funcția comparator cmp, înainte de a utiliza algoritmul de căutare binară!

8. Scrieți **o funcție generică de căutare** cu următorul antet:
void* cautare(**const void** *x, **const void** *t, **int** n, **int** d, **int** (*cmp)(**const void***,**const void** *))
Funcția trebuie să returneze un pointer generic către primul element din tabloul unidimensional **t** care este strict mai mare decât valoarea **x** (în raport cu un anumit criteriu de comparație) sau pointerul NULL dacă nu există nici un astfel de element în tablou. Tabloul unidimensional **t** este format din **n** elemente, fiecare având dimensiunea **d** octeți. Funcția comparator **cmp** implementează criteriul de comparație astfel: returnează valoarea -1 dacă primul argument este strict mai mic decât al doilea, 0 dacă argumentele sunt egale sau 1 dacă primul argument este strict mai mare decât al doilea. Folosind apeluri utile ale funcției **cautare**, scrieți o funcție care verifică dacă un tabloul unidimensional de numere întregi este sortat descrescător sau nu.
Rezolvare:
În cadrul funcției vom compara valoarea elementului curent **pcrt** din tablou cu valoarea căutată **x**, folosind funcția comparator. Dacă valoarea elementului curent este strict mai mare, atunci vom returna adresa sa. Funcția va returna valoarea NULL în cazul în care, după parcurgerea completă a tabloului, nu a fost găsit niciun element cu proprietatea cerută.

```
void* cautare(const void *x, const void *t, int n, int d,
int (*cmp)(const void* ,const void *))
{
    int i;
    char *pcrt;
    for(i = 0; i < n; i++)
    {
        pcrt = (char *)t + i*d;
        if(cmp(pcrt, x) > 0)
        return pcrt;
    }
    return NULL;
}
```

Pentru a verifica dacă un tablou unidimensional de numere întregi este sortat descrescător sau nu vom testa, pentru fiecare element din tablou, faptul că în dreapta sa nu există nici un element strict mai mare decât el.

```
int verif_sortat(int v[], int n)
{
    int i;
    for(i = 0; i < n-1; i++)
    if(cautare(v+i, v+i+1, n-i-1, sizeof(int), cmpIntregi) != NULL)
    return 0;
    return 1;
}
```

Observați faptul că în funcția verif_sortat nu am efectuat conversii explicite ale pointerilor generici spre pointeri către întregi! De ce?

Probleme propuse

1. Scrieți o funcție cu număr variabil de parametri care să concateneze mai multe șiruri de caractere, parcurgând o singură dată lista parametrilor variabili.

2. Scrieți o funcție cu număr variabil de parametri care să determine prefixul comun de lungime maximă al mai multor șiruri de caractere.

3. Scrieți o funcție cu număr variabil de parametri care să returneze numărul de apariții ale unui număr întreg dat într-un șir de numere întregi. Folosind apeluri utile ale funcției definite anterior, scrieți o funcție care verifică dacă 4 numere întregi sunt distincte sau nu.

4. Scrieți o funcție cu număr variabil de parametri care să creeze un șir de caractere din mai multe caractere date.

```
/*OBS:
- În funcțiile cu parametri variabili, tipul char este promovat automat la int.
*/

#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>

//Exemplul 1: functia primeste numarul de caractere, urmate de cele n caractere date individual
char *creeazaSir(int n, ...)
{
   va_list lista;
   va_start(lista, n);// precizeaza ca lista incepe dupa n

   char *sir = malloc((n + 1) * sizeof(char));
   if (sir == NULL) {
      va_end(lista);
      return NULL;
   }

   for (int i = 0; i < n; i++) {
      sir[i] = (char)va_arg(lista, int);  // char → int în varargs
   }
```

```
    sir[n] = '\0';

    va_end(lista);
    return sir;
}

int main(void)
{
    char *s = creeazaSir(6, 'E', 'x', 'a', 'm', 'e', 'n');

    //Atentie la numarul elementelor din lista!
    //char *s = creeazaSir(5, 'E', 'x', 'a', 'm', 'e', 'n');
    //char *s = creeazaSir(10, 'E', 'x', 'a', 'm', 'e', 'n');
    if (s) {
        printf("%s\n", s);
        free(s);// pentru ca am folosit alocare dinamica
    }

    return 0;
}


// Exemplul 2: funcția cu număr variabil de parametri creează un șir de caractere și se oprește la un caracter special (de exemplu '\0' sau '#') care este precizat ca argument fix
//Metoda:
// - Funcția nu primește numărul de caractere
// - Parametrii variabili sunt citiți până la întâlnirea unui caracter stop
// - Se face o primă parcurgere pentru a afla lungimea
// - Se alocă exact cât trebuie
// - Se face a doua parcurgere pentru copiere

//va_copy este necesar pentru ca va_list nu poate fi parcurs de două ori fără copiere
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>

char *creeazaSirStop(char stop, ...)
{
    va_list lista_1, lista_2;
    va_start(lista_1, stop);

    // Prima parcurgere – numărăm caracterele
    int count = 0;
    int c;
```

```
    va_copy(lista_2, lista_1);

    while ((c = va_arg(lista_2, int)) != stop)
        count++;

    va_end(lista_2);

    // Alocare spatiu pentru toate caracterele +1 pentru marcator de sfarsit de sir
    char *sir = malloc((count + 1) * sizeof(char));
    if (!sir) {
        va_end(lista_1);
        return NULL;
    }

    // A doua parcurgere – copiere
    for (int i = 0; i < count; i++) {
        sir[i] = (char)va_arg(lista_1, int);
    }

    sir[count] = '\0';//marcator de sfarsit de sir
    va_end(lista_1);
    return sir;
}

int main(void)
{
    //lista de argumente incepe si se termina cu caracterul special:
    char *s = creeazaSirStop('\0', 'A', 'n', 'a', '\0');
    /*sau incepe cu caracterul special si contine inca o data caracterul special urmat de alte
caractere, care nu vor fi citite */
    //char *s = creeazaSirStop('\0', 'A', 'n', 'a', '\0', 'c', 'o', 'n', 'd', 'a');

    if (s) {
        printf("%s\n", s);
        free(s);
    }

    return 0;
}
```

```
// Exemplul 3: varianta fara alocare dinamica
//nu mai parcurgem lista de argumente de doua ori
//șirul este stocat într-un buffer static; la fiecare apel, conținutul anterior este suprascris (deci folosim static!)
#include <stdio.h>
#include <stdarg.h>

#define MAX 100

char *creeazaSirStatic(char stop, ...)
{
   static char buffer[MAX];
   int i = 0;

   va_list litere;
   va_start(litere, stop);

   int c;
   while ((c = va_arg(litere, int)) != stop && i < MAX - 1) {
      buffer[i++] = (char)c;
   }

   buffer[i] = '\0';

   va_end(litere);
   return buffer;
}

int main(void)
{
   char *s = creeazaSirStatic('#', 'T', 'e', 's', 't', '#');
   printf("%s\n", s);

   return 0;
}
```

5. Scrieți o funcție cu număr variabil de parametri care să creeze un tablou de numere întregi din mai multe numere întregi. Atenție, funcția trebuie să furnizeze și lungimea tabloului!

6. Scrieți o funcție generică care să verifice dacă două tablouri unidimensionale sunt egale sau nu. Două tablouri unidimensionale sunt egale dacă au același număr de elemente și elementele aflate pe aceeași poziție sunt egale.

7. Scrieți o funcție generică de căutare care să returneze un pointer generic către prima apariție a unei valori x în tabloul unidimensional t format din n elemente, fiecare element având dimensiunea d octeți, sau pointerul $NULL$ dacă valoarea x nu se găsește în tablou. Scrieți o funcție care să afișeze, folosind apeluri utile ale funcției anterioare, toate pozițiile pe care apare un număr întreg x într-un tabloul v format din n numere întregi sau mesajul "$Valoare\ inexistentă$!" dacă numărul x nu se găsește în tablou.

8. Scrieți o funcție generică care să înlocuiască într-un tablou unidimensional toate aparițiile unei valori x cu o valoare y. Atenție la modul în care valoarea y este copiată în locul valorii x!

```
/*
Scrieti o funcție generică care să înlocuiască într-un tablou unidimensional toate aparițiile unei
valori x cu o valoare y.
Adaugati optiunea de a contoriza inlocuirile efectuate.

Obs:
- o funcție generică funcționează pentru orice tip de date (int, double, struct etc.)

- funcția generică are ca parametri:
      - tabloul (tratat ca un bloc de memorie (void *)),
      - numarul de elemente ale tabloului,
      - dimensiunea unui element (OBLIGATORIU)
      - adresa lui x si adresa lui y

- functia generica lucreaza cu memorie bruta (void*), deci este independenta de faptul ca
tabloul e static sau dinamic => funcția generică NU trebuie modificată pentru tablouri dinamice
— doar modul de apel diferă.

- compararea se face byte-cu-byte (memcmp);
- inlocuirea se face prin copierea conținutului lui y (memcpy) si nu a pointerului, deci NU prin
atribuire (NU scriem p[i] = y; // copiaza adresa, nu valoarea)

*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
   int a, b;
} Fractie;
```

```
void inlocuire(void *tablou, int n, int dim, const void *x, const void *y)
{
   char *p = (char *)tablou;

   for (int i = 0; i < n; i++)
      if (memcmp(p + i * dim, x, dim) == 0)
         memcpy(p + i * dim, y, dim);  // copiaza valoarea lui y
}

int inlocuire_nr(void *tablou, int n, int dim, const void *x, const void *y)
{
   char *p = (char *)tablou;
   int nrx = 0;
   for (int i = 0; i < n; i++)
      if (memcmp(p + i * dim, x, dim) == 0)
      {
         memcpy(p + i * dim, y, dim);  // copiaza valoarea lui y
         nrx++; //contorizeaza inlocuirile
      }
   return nrx;
}

int main(void)
{
   /*
   //Exemplul 1: Vector (static) de intregi:
   int v[] = {1, 2, 3, 2, 4, 2};
   int x = 2;
   int y = 9;
   int n = sizeof(v) / sizeof(v[0]);

   //int nr = inlocuire_nr(v, n, sizeof(int), &x, &y);
   //printf("S-au efectuat %d inlocuiri.\n", nr);

   inlocuire(v, n, sizeof(int), &x, &y);

   //afisare:
   for (int i = 0; i < n; i++)
      printf("%d ", v[i]);
   */
```

```
/*
   //Exemplul 2: Vector de intregi in alocare dinamica:
   /*Functia lucreaza cu memorie bruta (void*), deci este independenta de faptul ca tabloul e
static sau dinamic. */
   int n, x, y, *v;
   scanf("%d %d %d", &n, &x, &y);
   v = (int*)malloc(n*sizeof(int));
   if (v == NULL)
      return 1;

   for (int i = 0; i < n; i++)
      scanf("%d", (v + i));

   //int nr = inlocuire_nr(v, n, sizeof(int), &x, &y);
   //printf("S-au efectuat %d inlocuiri.\n", nr);

   inlocuire(v, n, sizeof(int), &x, &y);

   for (int i = 0; i < n; i++)
      printf("%d ", v[i]);

   free(v);//eliberare memorie
   */

   /*
   //Exemplul 3: Vector de numere reale - in alocare statica:
   float v[] = {1, 2.3, 3.14, 2.3, 4.06, 2.1};
   float x = 2.3;
   float y = 9;
   int n = sizeof(v) / sizeof(v[0]);

   //int nr = inlocuire_nr(v, n, sizeof(float), &x, &y);
   //printf("S-au efectuat %d inlocuiri.\n", nr);

   inlocuire(v, n, sizeof(float), &x, &y);

   //afisare:
   for (int i = 0; i < n; i++)
      printf("%.2f ", v[i]);
   */
```

```
/*
 //Exemplul 4: Vector de numere reale (float sau double) in alocare dinamica:
 int n;
 float x, y, *v;
 scanf("%d %f %f", &n, &x, &y);
 v = (float*)malloc(n*sizeof(float));
 if (v == NULL)
    return 1;

 for (int i = 0; i < n; i++)
    scanf("%f", (v + i));

 //int nr = inlocuire_nr(v, n, sizeof(float), &x, &y);
 //printf("S-au efectuat %d inlocuiri.\n", nr);

 inlocuire(v, n, sizeof(float), &x, &y);

 for (int i = 0; i < n; i++)
    printf("%.3f ", v[i]);

 free(v);//eliberare memorie
 */

 /*
 //Exemplul 5: Vector (static) de char:
 //char v[] = {'e', 'r', 'e', '2', '0', '2', '6'};
 char v[] = "teleenciclopedia";
 char x = 'e';
 char y = '*';
 int n = sizeof(v) / sizeof(v[0]);

 //int nr = inlocuire_nr(v, n, sizeof(char), &x, &y);
 //printf("S-au efectuat %d inlocuiri.\n", nr);

 inlocuire(v, n, sizeof(char), &x, &y);

 //afisare:
 for (int i = 0; i < n; i++)
    printf("%c ", v[i]);
 */
```

```
/*
  //Exemplul 6: Vector de struct in alocare statica:
  int n;    scanf("%d", &n);
  Fractie v[n], x, y;
  for(int i = 0; i < n; i++)
        scanf("%d %d", &v[i].a, &v[i].b);

  printf("date care se inlocuiesc: 1/0\n");    x.a = 1, x.b = 0;
  printf("valori pentru inlocuire: 5/4\n");    y.a = 5, y.b = 4;

  //int nr = inlocuire_nr(v, n, sizeof(Fractie), &x, &y);
  //printf("S-au efectuat %d inlocuiri.\n", nr);

  inlocuire(v, n, sizeof(Fractie), &x, &y);

  //afisare:
  for (int i = 0; i < n; i++)
     printf("%d/%d ", v[i].a, v[i].b);
  */

  //Exemplul 7: Vector de struct in alocare dinamica
  size_t n = 3;
  Fractie *v = malloc(n * sizeof(Fractie));

  if (!v) return 1;

  v[0] = (Fractie){1, 2};
  v[1] = (Fractie){3, 4};
  v[2] = (Fractie){1, 2};

  Fractie x = {1, 2};
  Fractie y = {5, 4};

  //int nr = inlocuire_nr(v, n, sizeof(Fractie), &x, &y);
  //printf("S-au efectuat %d inlocuiri.\n", nr);

  inlocuire(v, n, sizeof(Fractie), &x, &y);

 for (int i = 0; i < n; i++)
     printf("%d/%d ", v[i].a, v[i].b);

  free(v);//eliberare memorie
  return 0;
}
```

```
//Varianta cu functie comparator cu memcmp
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int cmp_1(const void* a, const void* b, const int dim) {
	if(memcmp(a, b, dim) == 0) return 1;
	return 0;
}

int cmp_2(const void* a, const void* b, const int dim) {
	if(memcmp(a, b, dim) < 0) return 1;
	return 0;
}

void functie(void* vector, const int n, const int dim, const void* x, const void* y, int (*f)(const
void*, const void*, const int)) {
	char* v = (char*)vector;
	for(int i=0; i<n; i++)	{
		char* element_curent = v + (i * dim);
		if(f(element_curent, x, dim)) memcpy(v + (i * dim), y, dim);
	}
}

int main() {
	int n;	scanf("%d", &n);
	// lucrez cu tipul int:
	int x,y; scanf("%d %d", &x, &y);
	int dim = sizeof(int);
	int* v = (int*)malloc( n * dim);
	for(int i=0; i<n; i++)
		scanf("%d", v+i);

	//inlocuieste cu y toate valorile din v care sunt egale cu x
	functie(v, n, dim, &x, &y, cmp_1);

	//inlocuieste cu y toate valorile din v care sunt mai mici decat x
	functie(v, n, dim, &x, &y, cmp_2);

	for(int i=0; i<n; i++)
		printf("%d ", *(v+i));
	return 0;
}
```